Pi Frame — Low-Level Design (LLD)

> **Version:** 1.0 — initial LLD
> **Status:** Authoritative
> **Source documents:** `pi-frame-ux-requirements.md`, `pi-frame-hld.md`, `pi-frame-overlay-mockup.html`, `pi-frame-settings-mockup.html`

Preamble

Pass 1 — Requirements traceability

Every requirement ID from `pi-frame-ux-requirements.md` is listed below with the module/section that satisfies it and its v1/deferred status.

ID	Requirement summary	Module / section	Status
SH-01	Full-screen JPEG display	SlideshowPlayer, PhotoCache	v1 §S1
SH-02	Auto-advance interval	SlideshowPlayer, ConfigStore	v1 §S1
SH-03	Shuffle playlist	SlideshowPlayer (Fisher-Yates)	v1 §S1
SH-04	Crossfade / cut / slide transitions	SlideshowPlayer	v1 §S1
SH-05a	Paused indicator pip	SlideshowPlayer.draw_pip()	v1 §S2
SH-05b	Settings-gear button	OverlayUI	v1 §S2
SH-06	Clock overlay	ClockWidget	v1 §S1
SH-07	EXIF orientation correction	PhotoCache (PIL tag 274)	v1 §S1
SH-08	Blurred-background composite (fit)	PhotoCache fit-mode	v1 §S1
PS-01	Tap anywhere to show overlay	App (MOUSEBUTTONDOWN)	v1 §S2
PS-02	Play / pause button	OverlayUI	v1 §S2
PS-03	Previous / next buttons	OverlayUI → SlideshowPlayer	v1 §S2
PS-04	Swipe left/right to skip	App swipe detection	v1 §S2
PS-05	Overlay auto-dismissal timer	OverlayUI / App	v1 §S2
PS-06	Brightness slider (overlay)	OverlayUI, BacklightController, VerticalSlider	v1 §S2
PB-01	Settings panel (gear icon)	SettingsPanel	v1 §S4
PB-02	Slideshow settings section	SettingsPanel §Slideshow	v1 §S4

ID	Requirement summary	Module / section	Status
PB-03	Display settings section	SettingsPanel §Display	v1 §S5
PB-04	Wi-Fi settings section	SettingsPanel §Wi-Fi, WifiManager	v1 §S7
PB-05	Photo metadata in overlay	deferred	post-v1
BL-01	Backlight brightness control	BacklightController (sysfs)	v1 §S2
BL-02	Brightness persisted to TOML	ConfigStore [display].brightness	v1 §S3
BL-03	Brightness visible while dragging	OverlayUI (live update)	v1 §S2
BL-04	Sleep dims screen to 0	SleepScheduler → BacklightController	v1 §S5
BL-05	Ambient light adjustment	deferred	post-v1
KB-01	On-screen keyboard appears for text fields	Keyboard, TextInput, KEYBOARD state	v1 §S6
KB-02	Alpha / numeric / extended layers	Keyboard layer switching	v1 §S6
KB-03	Shift key (single-shot)	Keyboard shift state	v1 §S6
KB-04	Backspace key	Keyboard → TextInput	v1 §S6
KB-05	Done key dismisses keyboard	Keyboard → SETTINGS state	v1 §S6
DS-01	TOML config file	ConfigStore	v1 §S3
DS-02	Extra slide transitions	deferred	post-v1
DS-03	Sleep schedule	SleepScheduler	v1 §S5
DS-04	Timezone picker	System section, ScrollPicker	v1 §S5
DS-05	Info bar on photos	deferred	post-v1
WF-01	Wi-Fi network list	WifiManager.scan(), WifiListItem	v1 §S7
WF-02	Connect to Wi-Fi with password	WifiManager.connect(), Keyboard	v1 §S7
WF-03	Forget saved network	WifiManager.forget(), ConfirmDialog	v1 §S7
WF-04	Wi-Fi status indicator	WifiManager.status(), OverlayUI icon	v1 §S7
WF-05	First-run onboarding	deferred	post-v1
SY-01	OneDrive sync integration	SyncService	v1 §S9
SY-02	Sync status in Settings	SettingsPanel §Sync row	v1 §S9
SY-03	Manual sync trigger	SettingsPanel → SyncService.trigger()	v1 §S9

ID	Requirement summary	Module / section	Status
SY-04	OTA update	SettingsPanel §System, App.check_update()	v1 §S8

Pass 2 — HLD section coverage

HLD section	LLD section
§2 State machine	§2.2 AppState/AppEvent + §3.1 App
§3 Rendering pipeline	§3.2 SlideshowPlayer, §3.3 PhotoCache, layer-stack §3.1
§4 Modules	§3.1–3.12
§5 Widgets	§4.1–4.11
§6 Config / TOML schema	§3.10 ConfigStore
§7 Background services	§3.8 SyncService, §3.9 SleepScheduler, §3.11 BacklightController
§8 Hardware interfaces	§3.11 BacklightController, §3.12 WifiManager
§9 Implementation stages	§6
§10 Open items	§7

1. Project layout

```
digital-frame/
├── slideshow.py            # Entry point: from piframe.app import App;
App().run()
├── piframe/                # Main application package
│   ├── __init__.py
│   ├── app.py              # App class, main loop, state machine
│   ├── photo_cache.py      # PhotoCache: PIL composite → pygame.Surface,
LRU
│   ├── sync_service.py     # SyncService: daemon thread wrapping framesync
│   ├── overlay_ui.py       # OverlayUI: transient player controls +
brightness
│   ├── settings_panel.py   # SettingsPanel: all settings sections
│   ├── keyboard.py         # On-screen keyboard widget (full-row)
│   ├── backlight.py        # BacklightController: /sys/class/backlight
sysfs
│   ├── wifi_manager.py     # WifiManager, WifiNetwork, WifiStatus,
WifiResult
│   ├── config_store.py     # ConfigStore: TOML read/write with debounce
│   ├── clock_widget.py     # ClockWidget: time+date, daemon ticker thread
│   ├── sleep_scheduler.py  # SleepScheduler: sleep/wake daemon thread
│   ├── assets.py           # Assets singleton: font+icon loader
│   ├── types.py            # Shared dataclasses and enums
│   ├── widgets/
│   │   ├── __init__.py     # Re-exports all widget classes
```

```
│   │   ├── base.py          # Widget ABC
│   │   ├── toggle.py        # Toggle (animated 50×28px)
│   │   ├── vertical_slider.py  # VerticalSlider (brightness, 4px track)
│   │   ├── segmented_control.py# SegmentedControl (2–4 segments)
│   │   ├── scroll_picker.py    # ScrollPicker (7-row windowed list)
│   │   ├── time_picker.py      # TimePicker (HH MM pills + popup)
│   │   ├── wifi_list_item.py   # WifiListItem (ssid + signal + lock)
│   │   ├── confirm_dialog.py   # ConfirmDialog (480×240 modal)
│   │   ├── nav_item.py         # NavItem (sidebar navigation row)
│   │   └── text_input.py       # TextInput (single-line, keyboard
attachment)
│   └── assets/
│       └── fonts/
│           ├── NotoSans-Regular.ttf
│           ├── NotoSans-Bold.ttf
│           └── MaterialIcons-Regular.ttf
├── framesync/              # OneDrive sync module (modifiable)
│   └── framesync.py        # sync_folder(), load_config(); used by
SyncService
├── config.toml             # Not committed (secrets)
├── config.toml.example     # Committed template
├── tests/                  # pytest unit + headless + integration tests
│   ├── conftest.py
│   ├── image_utils.py
│   ├── golden/
│   └── test_*.py
├── eng/
│   └── install.sh          # Deployment script (idempotent)
└── docs/
    ├── pi-frame-ux-requirements.md
    ├── pi-frame-hld.md
    ├── pi-frame-overlay-mockup.html
    ├── pi-frame-settings-mockup.html
    └── pi-frame-lld.md     # This document
```

---

2. Shared types & constants

2.1 Screen constants (`piframe/types.py`)

```
SCREEN_W        = 1280
SCREEN_H        = 800
RIGHT_COL_W     = 80      # overlay right column
SIDEBAR_W       = 333     # settings sidebar (≈26 % of screen width)
BOTTOM_BAR_H    = 88      # overlay bottom bar
SETTINGS_CONTENT_X = SIDEBAR_W          # 333
SETTINGS_CONTENT_W = SCREEN_W - SIDEBAR_W  # 947
FPS             = 30
TRANS_DURATION  = 0.5     # seconds for crossfade / slide
```

```
OVERLAY_DISMISS = 5.0     # seconds before overlay auto-dismisses
WAKE_GRACE      = 30.0    # seconds of grace after tap-to-wake
```

2.2 State machine enums (`piframe/types.py`)

```python
from enum import Enum, auto

class AppState(Enum):
    SLIDESHOW = auto()
    OVERLAY   = auto()
    SETTINGS  = auto()
    KEYBOARD  = auto()
    SLEEPING  = auto()

class AppEvent(Enum):
    SLEEP           = auto()
    WAKE            = auto()
    SYNC_COMPLETE   = auto()
    OVERLAY_DISMISS = auto()
```

State-transition table (all guards listed):

From	To	Trigger	Guard
SLIDESHOW	OVERLAY	MOUSEBUTTONDOWN	—
SLIDESHOW	SLEEPING	EVT_SLEEP	—
OVERLAY	SLIDESHOW	dismiss timer	not paused
OVERLAY	SLIDESHOW	tap outside controls	—
OVERLAY	SETTINGS	gear-icon tap	—
OVERLAY	SLEEPING	EVT_SLEEP	—
SETTINGS	SLIDESHOW	"Back to frame" tap	—
SETTINGS	KEYBOARD	TextInput.on_focus()	—
SETTINGS	SLEEPING	EVT_SLEEP	—
KEYBOARD	SETTINGS	Done key / tap outside keyboard	—
KEYBOARD	SLEEPING	EVT_SLEEP	—
SLEEPING	OVERLAY	MOUSEBUTTONDOWN	tap-to-wake
SLEEPING	SLIDESHOW	EVT_WAKE	scheduled wake

2.3 Custom pygame event IDs (`piframe/types.py`)

```
import pygame

EVT_SYNC_COMPLETE = pygame.USEREVENT + 1  # posted by SyncService
EVT_SLEEP         = pygame.USEREVENT + 2  # posted by SleepScheduler
EVT_WAKE          = pygame.USEREVENT + 3  # posted by SleepScheduler
EVT_UPDATE_RESULT = pygame.USEREVENT + 4  # posted by OTA check thread
EVT_WIFI_RESULT   = pygame.USEREVENT + 5  # posted by WifiManager threads
```

2.4 Dataclasses (`piframe/types.py`)

```
from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class SyncStatus:
    last_sync_time: datetime | None = None
    photo_count:    int             = 0
    in_progress:    bool            = False
    last_error:     str | None      = None

@dataclass
class WifiNetwork:
    ssid:     str
    security: str   # "" → open network
    signal:   int   # 0–100

    @property
    def signal_level(self) -> int:
        """Returns 0, 1, or 2 for icon strength tier."""
        if self.signal >= 67: return 2
        if self.signal >= 34: return 1
        return 0

@dataclass
class WifiStatus:
    connected:  bool
    ssid:       str   # "" if disconnected
    ip_address: str   # "" if disconnected

@dataclass
class WifiResult:
    operation: str          #
"scan"|"connect"|"forget"|"disconnect"|"status"
    success:   bool
    data:      object | None = None
    error:     str | None    = None

@dataclass
class UpdateResult:
    available:   bool
```

```
    tag_name:    str  = ""
    tarball_url: str  = ""
    error:       str | None = None
```

2.5 Colour palette (`piframe/types.py`)

All colours are 4-tuples `(R, G, B, A)`.

```
COLOUR_SCRIM              = (  0,   0,   0, 140)
COLOUR_OVERLAY_BTN_BG     = (255, 255, 255,  30)
COLOUR_OVERLAY_BTN_BD     = (255, 255, 255,  51)
COLOUR_PROGRESS_BAR       = (255, 255, 255, 179)
COLOUR_SIDEBAR_BG         = ( 24,  24,  24, 255)
COLOUR_CONTENT_BG         = ( 17,  17,  17, 255)
COLOUR_NAV_ACTIVE_BG      = (255, 255, 255,  23)
COLOUR_TOGGLE_ON          = ( 55, 138, 221, 255)
COLOUR_TOGGLE_OFF         = ( 80,  80,  80, 255)
COLOUR_TOGGLE_THUMB       = (255, 255, 255, 255)
COLOUR_DESTRUCTIVE        = (242,  75,  74, 255)
COLOUR_CONNECTED          = ( 83,  74, 183, 255)
COLOUR_TEXT_PRIMARY       = (255, 255, 255, 255)
COLOUR_TEXT_SECONDARY     = (153, 153, 153, 255)
COLOUR_TEXT_CAPTION       = (102, 102, 102, 255)
COLOUR_DIVIDER            = (255, 255, 255,  20)
COLOUR_SLIDER_TRACK       = (255, 255, 255,  51)
COLOUR_SLIDER_THUMB       = (255, 255, 255, 255)
COLOUR_SLIDER_FILL        = (255, 255, 255, 179)
COLOUR_KEY_BG             = ( 50,  50,  50, 255)
COLOUR_KEY_BG_SPECIAL     = ( 80,  80,  80, 255)
COLOUR_KEY_BG_ACTIVE      = (100, 100, 100, 255)
COLOUR_SCROLL_PICKER_HL   = (255, 255, 255,  25)
COLOUR_PILL_BG            = ( 50,  50,  50, 255)
COLOUR_PILL_BORDER        = (255, 255, 255,  51)
COLOUR_DIALOG_BG          = ( 30,  30,  30, 245)
COLOUR_DIALOG_BORDER      = (255, 255, 255,  30)
COLOUR_BTN_PRIMARY        = ( 55, 138, 221, 255)
COLOUR_BTN_SECONDARY      = ( 60,  60,  60, 255)
COLOUR_WIFI_STRENGTH_0    = ( 80,  80,  80, 255)
COLOUR_WIFI_STRENGTH_1    = (255, 255, 255, 255)
COLOUR_CLOCK_TEXT         = (255, 255, 255, 220)
COLOUR_OVERLAY_SCRIM      = (  0,   0,   0,  90)
```

2.6 Font & icon constants (`piframe/assets.py`)

```
FONT_SIZE_CLOCK       = 48
FONT_SIZE_HEADING     = 24
FONT_SIZE_BODY        = 18
FONT_SIZE_NAV         = 20
FONT_SIZE_SECONDARY   = 14
```

```
FONT_SIZE_KEY         = 16

ICON_SIZE_NORMAL      = 24
ICON_SIZE_OVERLAY     = 32

# Material Icons codepoints
IC_SETTINGS     = "\ue8b8"
IC_PLAY         = "\ue037"
IC_PAUSE        = "\ue034"
IC_SKIP_PREV    = "\ue044"
IC_SKIP_NEXT    = "\ue043"
IC_ARROW_BACK   = "\ue5d5"
IC_ARROW_FWD    = "\ue5dc"
IC_INFO         = "\ue87d"
IC_WIFI         = "\ue8f4"
IC_WIFI_OFF     = "\ue8f5"
IC_SYNC         = "\ue1d8"
IC_CLOSE        = "\ue5cd"
IC_CHECK        = "\ue876"
IC_CHEVRON_L    = "\ue5c4"
IC_CHEVRON_R    = "\ue5c8"
IC_EXPAND_MORE  = "\ue5cf"
IC_EXPAND_LESS  = "\ue5ce"
IC_BRIGHTNESS   = "\ue896"
IC_SCHEDULE     = "\ue8b5"
IC_PERSON       = "\ue7ef"
IC_DELETE       = "\ue872"
```

---

3. Module designs

3.1 `piframe/app.py` — `App`

Purpose

Top-level orchestrator. Owns the pygame window, main loop, state machine, event dispatch, draw stack, and module lifecycle.

Constructor

```
class App:
    def __init__(self) -> None:
        pygame.init()
        pygame.freetype.init()
        self._screen:    pygame.Surface        #
pygame.display.set_mode(...)
        self._clock:     pygame.time.Clock
        self._state:     AppState
        self._config:    ConfigStore
        self._assets:    Assets
        self._player:    SlideshowPlayer
```

```
        self._cache:     PhotoCache
        self._overlay:   OverlayUI
        self._settings:  SettingsPanel
        self._keyboard:  Keyboard
        self._clock_w:   ClockWidget
        self._backlight: BacklightController
        self._sync:      SyncService
        self._sleep:     SleepScheduler
        self._wifi:      WifiManager
        # Swipe-detection state
        self._swipe_start_pos:  tuple[int,int] | None
        self._swipe_start_time: float | None
        # Active modal dialog (may be None)
        self._dialog:    ConfirmDialog | None
```

Initialisation sequence (in `__init__`)

1. `pygame.display.set_mode((SCREEN_W, SCREEN_H), pygame.FULLSCREEN | pygame.NOFRAME)`
2. `pygame.mouse.set_visible(False)`
3. Load `config.toml` via `ConfigStore`.
4. Load `Assets` singleton (fonts, icons).
5. Construct `BacklightController`; apply `config.display.brightness`.
6. Construct `PhotoCache`, `SlideshowPlayer`.
7. Construct `ClockWidget`; start ticker thread.
8. Construct `OverlayUI`, `SettingsPanel`, `Keyboard`.
9. Construct `WifiManager`.
10. Construct `SyncService`; start daemon thread.
11. Construct `SleepScheduler`; start daemon thread.
12. Write PID to `/tmp/slideshow.pid`.
13. Set `_state = AppState.SLIDESHOW`.

`run()` — main loop

```
def run(self) -> None:
    while True:
        dt = self._clock.tick(FPS) / 1000.0
        self._process_pygame_events()
        self._drain_custom_events()
        self._config.tick(time.monotonic())
        if self._state == AppState.SLEEPING:
            time.sleep(0.25)
            continue
        self._update(dt)
        self._draw()
        pygame.display.flip()
```

`_process_pygame_events()`

- `QUIT` → `self._shutdown()`
- `KEYDOWN K_ESCAPE` → `self._shutdown()`
- `MOUSEBUTTONDOWN`: record `_swipe_start_pos` + `_swipe_start_time`; if state==SLEEPING → tap-to-wake
- `MOUSEBUTTONUP`: call `_classify_pointer_up(event.pos)`
- `MOUSEMOTION` with `event.buttons[0]`: if state==OVERLAY → `_overlay.on_drag(event.pos)`

`_classify_pointer_up(pos)`

```
if _swipe_start_pos is None: return
dx = pos[0] - _swipe_start_pos[0]
dy = pos[1] - _swipe_start_pos[1]
elapsed = time.monotonic() - _swipe_start_time
_swipe_start_pos = None
if abs(dx) > 60 and abs(dy) < 40 and elapsed < 0.4:
    if dx < 0: _player.skip_next()
    else:      _player.skip_prev()
    return
# Tap dispatch
_dispatch_tap(pos)
```

`_dispatch_tap(pos)`

State	Tap area	Action
SLIDESHOW	anywhere	→ OVERLAY; `_overlay.show()`
OVERLAY	gear button	→ SETTINGS; `_settings.open()`
OVERLAY	controls	delegate to `_overlay.on_tap(pos)`
OVERLAY	outside controls	→ SLIDESHOW; `_overlay.hide()`
SETTINGS	"Back to frame"	→ SLIDESHOW
SETTINGS	TextInput	`field.on_tap(pos)` → `_state = KEYBOARD`
KEYBOARD	outside keyboard rect	→ SETTINGS; `_keyboard.hide()`
SLEEPING	anywhere	tap-to-wake: restore brightness, `_state = OVERLAY`, start 5 s dismiss timer

`_drain_custom_events()`

Checks `pygame.event.get()` for custom event types:

Event	Action
EVT_SYNC_COMPLETE	`_player.rescan()`; `_settings.refresh_sync_status()`
EVT_SLEEP	`_enter_sleep()`
EVT_WAKE	`_exit_sleep()`
EVT_UPDATE_RESULT	`_settings.on_update_result(evt.result)`
EVT_WIFI_RESULT	`_settings.on_wifi_result(evt.result)`

`_enter_sleep()` / `_exit_sleep()`

```
    def _enter_sleep(self) -> None:
        self._backlight.set_brightness(0)
        self._state = AppState.SLEEPING

    def _exit_sleep(self) -> None:
        self._backlight.set_brightness(self._config.display.brightness)
        self._state = AppState.OVERLAY
        self._overlay.show()
        self._sleep.set_grace(time.monotonic() + WAKE_GRACE)
```

`_update(dt)`

Calls `.update(dt)` on whichever modules are active:

- Always: `_player.update(dt)`, `_clock_w.update(dt)`
- If OVERLAY: `_overlay.update(dt)` → if `_overlay.dismissed` → `_state = SLIDESHOW`
- If SETTINGS or KEYBOARD: `_settings.update(dt)`
- If KEYBOARD: `_keyboard.update(dt)`

`_draw()` — layer stack

```
1. _player.draw(screen)            — photo + active transition
2. _clock_w.draw(screen)           — if show_clock and state in {SLIDESHOW,
OVERLAY}
3. _player.draw_pip(screen)        — if paused and state == SLIDESHOW
4. _overlay.draw(screen)           — if state == OVERLAY
5. _settings.draw(screen)          — if state in {SETTINGS, KEYBOARD}
6. _keyboard.draw(screen)          — if state == KEYBOARD
7. draw_dialog(screen)             — if _dialog is not None
```

Steps 1–3 are skipped in SETTINGS / KEYBOARD states.

`restart()` and `_shutdown()`

```
    def restart(self) -> None:
        self._cleanup()
        import os, sys
        env = os.environ.copy()
        env["XDG_RUNTIME_DIR"] = "/run/user/1000"
        env["WAYLAND_DISPLAY"]  = "wayland-0"
        os.execve(sys.executable, [sys.executable] + sys.argv, env)

    def _shutdown(self) -> None:
        self._cleanup()
        pygame.quit()
        sys.exit(0)

    def _cleanup(self) -> None:
        self._sync.stop()
        self._sleep.stop()
        self._clock_w.stop()
        self._config.flush_now()
```

3.2 `piframe/photo_cache.py` — `PhotoCache`

Purpose

Converts image files on disk into composited, EXIF-corrected `pygame.Surface` objects. Maintains an LRU in-memory cache capped at 6 surfaces. Writes cached surfaces to disk as PNG for fast reload.

Constants

```
MAX_CACHE      = 6
_CACHE_VERSION = 2   # bump when rendering pipeline changes
BLUR_RADIUS    = 40
```

Data members

Member	Type	Description
`_cache_dir`	`Path`	Disk cache directory
`_fit_mode`	`str`	`"fit"` or `"fill"`
`_mem_cache`	`dict[str, pygame.Surface]`	In-memory LRU store
`_order`	`list[str]`	Most-recently-used last

`__init__(cache_dir, fit_mode)`

```
def __init__(self, cache_dir: str | Path, fit_mode: str = "fit") -> None:
```

Creates `cache_dir` if absent.

`get(path: Path) -> pygame.Surface`

1. Compute `key = f"{path.stem}_{_fit_mode}_v{_CACHE_VERSION}"`.
2. Check `_mem_cache`; if hit → move key to end of `_order`; return surface.
3. Check `cache_dir / f"{key}.png"`; if exists → load with `pygame.image.load()`; put into mem-cache; return.
4. Composite the image (see algorithms below); save to disk PNG; put into mem-cache; return.

`_put(key, surface)`

```
if len(_mem_cache) >= MAX_CACHE:
    evict = _order.pop(0)
    del _mem_cache[evict]
_mem_cache[key] = surface
_order.append(key)
```

`set_fit_mode(mode: str)`

Updates `_fit_mode`. Does **not** invalidate the disk cache (old entries remain accessible by their full key; new key includes new mode). No memory invalidation needed — new surfaces will be requested under the new key.

`invalidate_disk()`

Deletes all `.png` files in `_cache_dir`. Used when a breaking rendering change requires cache purge without bumping `_CACHE_VERSION` at runtime.

Fit-mode composite algorithm

```
Input: path (JPEG file)
Output: pygame.Surface (SCREEN_W × SCREEN_H, RGBA)

1.  img = PIL.Image.open(path)
2.  exif_val = img.getexif().get(274, 1)
3.  Apply EXIF orientation (see §3.2 EXIF table).
4.  img = img.convert("RGB")
5.  # Foreground — letterboxed
    scale = min(SCREEN_W/img.width, SCREEN_H/img.height)
    fg_w, fg_h = int(img.width*scale), int(img.height*scale)
    fg_img = img.resize((fg_w, fg_h), PIL.Image.LANCZOS)
6.  fg_surf = pygame.Surface((SCREEN_W, SCREEN_H), pygame.SRCALPHA)
```

```
    fg_surf.fill((0, 0, 0, 255))
    paste_x = (SCREEN_W - fg_w) // 2
    paste_y = (SCREEN_H - fg_h) // 2
    fg_surf.blit(pygame_image_from_pil(fg_img), (paste_x, paste_y))
7.  # Background — blurred cover crop
    scale_bg = max(SCREEN_W/img.width, SCREEN_H/img.height)
    bg_w, bg_h = int(img.width*scale_bg), int(img.height*scale_bg)
    bg_img = img.resize((bg_w, bg_h), PIL.Image.LANCZOS)
    crop_x = (bg_w - SCREEN_W) // 2
    crop_y = (bg_h - SCREEN_H) // 2
    bg_img = bg_img.crop((crop_x, crop_y, crop_x+SCREEN_W,
crop_y+SCREEN_H))
    bg_img = bg_img.filter(PIL.ImageFilter.GaussianBlur(BLUR_RADIUS))
8.  out = pygame.Surface((SCREEN_W, SCREEN_H), pygame.SRCALPHA)
    out.blit(pygame_image_from_pil(bg_img), (0, 0))
    out.blit(fg_surf, (0, 0))
9.  return out
```

Fill-mode composite algorithm

```
1.  img = PIL.Image.open(path); EXIF correct; convert("RGB")
2.  scale = max(SCREEN_W/img.width, SCREEN_H/img.height)
3.  w, h = int(img.width*scale), int(img.height*scale)
4.  img = img.resize((w, h), PIL.Image.LANCZOS)
5.  crop_x = (w - SCREEN_W) // 2
    crop_y = (h - SCREEN_H) // 2
    img = img.crop((crop_x, crop_y, crop_x+SCREEN_W, crop_y+SCREEN_H))
6.  return pygame_image_from_pil(img)  # RGB surface, no alpha needed
```

EXIF orientation (tag 274)

Value	Transform
1	no-op
2	`img.transpose(FLIP_LEFT_RIGHT)`
3	`img.rotate(180)`
4	`img.transpose(FLIP_TOP_BOTTOM)`
5	`img.transpose(TRANSPOSE)`
6	`img.transpose(ROTATE_270)`
7	`img.transpose(TRANSVERSE)`
8	`img.transpose(ROTATE_90)`

`pygame_image_from_pil(pil_img) -> pygame.Surface` **(module-level helper)**

```
    raw  = pil_img.tobytes()
    mode = pil_img.mode          # "RGB" or "RGBA"
    size = pil_img.size
    return pygame.image.frombuffer(raw, size, mode).convert_alpha()
```

3.3 `SlideshowPlayer` (in `piframe/app.py`)

Manages the active playlist, transitions, and draws to the screen surface.

Data members

Member	Type	Description
`_playlist`	`list[Path]`	Shuffled or sorted photo list
`_index`	`int`	Current position in `_playlist`
`_current_surface`	`pygame.Surface | None`	Displayed frame
`_trans_incoming`	`pygame.Surface | None`	Copy used during transition
`_trans_progress`	`float`	0.0 → 1.0
`_trans_start`	`float`	`time.monotonic()` at start of transition
`_trans_mode`	`str`	`"crossfade"` | `"cut"` | `"slide"`
`_direction`	`int`	+1 forward, −1 backward
`_in_transition`	`bool`	—
`_paused`	`bool`	—
`_slide_elapsed`	`float`	Seconds since last advance
`_interval`	`float`	From config
`_photo_dir`	`Path`	Watched directory
`_cache`	`PhotoCache`	—

`__init__(photo_dir, cache, config)`

1. `rescan()` to populate `_playlist`.
2. `_preload_current()`.

`rescan()`

```
    files = sorted(_photo_dir.glob("*.jpg")) +
    sorted(_photo_dir.glob("*.jpeg"))
    files += sorted(_photo_dir.glob("*.png"))
```

```
    if config.slideshow.shuffle:
        _fisher_yates(files)
    _playlist = files
    _index = 0
```

`_fisher_yates(lst)`

```
    for i in range(len(lst) - 1, 0, -1):
        j = random.randint(0, i)
        lst[i], lst[j] = lst[j], lst[i]
```

`update(dt: float)`

```
    if _paused or not _playlist: return

    if _in_transition:
        progress = (time.monotonic() - _trans_start) / TRANS_DURATION
        _trans_progress = min(1.0, progress)
        if _trans_progress >= 1.0:
            _commit_transition()
        return

    _slide_elapsed += dt
    if _slide_elapsed >= _interval:
        _slide_elapsed = 0.0
        advance()
```

`advance()` **and** `go_back()`

```
    def advance(self) -> None:
        _direction = +1
        _start_transition((_index + 1) % len(_playlist))

    def go_back(self) -> None:
        _direction = -1
        _start_transition((_index - 1) % len(_playlist))
```

`_start_transition(next_index)`

```
    _in_transition = True
    _trans_start   = time.monotonic()
    _trans_progress = 0.0
    next_surf = _cache.get(_playlist[next_index])
```

```
    if _trans_mode == "cut":
        _current_surface = next_surf
        _index = next_index
        _in_transition = False
        return
    _trans_incoming = next_surf.copy()
    _pending_index  = next_index
```

`_commit_transition()`

```
    _current_surface = _trans_incoming
    _trans_incoming.set_alpha(255)   # restore after alpha-blend use
    _index = _pending_index
    _trans_incoming = None
    _in_transition  = False
```

`draw(screen: pygame.Surface)`

```
    if not _current_surface: return
    screen.blit(_current_surface, (0, 0))
    if _in_transition and _trans_incoming:
        if _trans_mode == "crossfade":
            _trans_incoming.set_alpha(int(255 * _trans_progress))
            screen.blit(_trans_incoming, (0, 0))
        elif _trans_mode == "slide":
            cur_x  = int(-_direction * _trans_progress * SCREEN_W)
            next_x = int( _direction * (1.0 - _trans_progress) * SCREEN_W)
            screen.blit(_current_surface, (cur_x,  0))
            screen.blit(_trans_incoming,  (next_x, 0))
```

`draw_pip(screen)`

Draws a 26×26 px "paused" pill at `(12, 762)` when `_paused` is `True` and state is SLIDESHOW:

```
    pip_surf = pygame.Surface((26, 26), pygame.SRCALPHA)
    pygame.draw.rect(pip_surf, COLOUR_OVERLAY_BTN_BG, (0,0,26,26),
    border_radius=6)
    assets.icon(IC_PAUSE, 14).blit(pip_surf, (6, 6))
    screen.blit(pip_surf, (12, 762))
```

`skip_next()` / `skip_prev()`

Calls `advance()` / `go_back()` and resets `_slide_elapsed = 0`.

`toggle_pause()`

```
_paused = not _paused
```

3.4 `piframe/overlay_ui.py` — `OverlayUI`

Transient overlay shown in OVERLAY state. Owns the bottom bar (previous / play-pause / next), right column (settings gear, brightness slider), and the dismiss timer.

Data members

Member	Type	Description
`_visible`	`bool`	—
`_dismiss_at`	`float | None`	`time.monotonic()` of auto-dismiss
`_paused`	`bool`	Mirrors player state
`_brightness`	`int`	0–100
`_slider`	`VerticalSlider`	Brightness slider instance
`_dragging_slider`	`bool`	True during active drag
`dismissed`	`bool`	Polled by App

Layout constants

```
GEAR_CENTER    = (1240, 33)
GEAR_RECT      = pygame.Rect(1221, 14, 38, 38)
SUN_HI_CENTER  = (1240, 108)
SUN_LO_CENTER  = (1240, 744)
SLIDER_RECT    = pygame.Rect(1238, 130, 4, 588)  # track rect
BRIGHTNESS_LABEL_CENTER = (1240, 776)

PREV_RECT  = pygame.Rect(508,  732, 48, 48)
PLAY_RECT  = pygame.Rect(572,  728, 56, 56)
NEXT_RECT  = pygame.Rect(644,  732, 48, 48)
DISMISS_BAR = pygame.Rect(0, 0, 1280, 3)
```

`show()`

```
_visible   = True
dismissed  = False
_dismiss_at = time.monotonic() + OVERLAY_DISMISS if not _paused else None
```

`hide()`

```
_visible  = False
dismissed = True
```

`update(dt)`

```
if not _visible: return
if _dismiss_at and time.monotonic() >= _dismiss_at:
    hide()
```

`draw(screen)`

1. Draw semi-transparent scrim: fill `(0,0,SCREEN_W,SCREEN_H)` with `COLOUR_OVERLAY_SCRIM`.
2. Draw 3 px dismiss bar at y=0 with `COLOUR_PROGRESS_BAR`.
3. Draw right column: a. Gear button: filled rounded rect `COLOUR_OVERLAY_BTN_BG` + border `COLOUR_OVERLAY_BTN_BD`; draw settings icon at center. b. Sun-hi icon at `SUN_HI_CENTER`. c. `_slider.draw(screen)`. d. Sun-lo icon at `SUN_LO_CENTER`. e. Brightness percent label (14pt, `COLOUR_TEXT_PRIMARY`) at `BRIGHTNESS_LABEL_CENTER`.
4. Draw bottom bar: a. Previous button (48×48 rounded rect) at `PREV_RECT`. b. Play/Pause primary button (56×56) at `PLAY_RECT`. c. Next button (48×48) at `NEXT_RECT`. d. Draw appropriate icon: `IC_PAUSE` if playing, `IC_PLAY` if paused (32px).

`on_tap(pos) -> str | None`

Returns action string used by App to dispatch:

Hit test	Returns
`GEAR_RECT`	`"settings"`
`PREV_RECT`	`"prev"`
`PLAY_RECT`	`"play_pause"`
`NEXT_RECT`	`"next"`
`DISMISS_BAR`	`"dismiss"`
elsewhere	`None`

`on_drag(pos)`

If `SLIDER_RECT.collidepoint(pos)` or `_dragging_slider`:

- Compute new brightness from y-coordinate via `_y_to_brightness(pos[1])`.
- Clamp to [0, 100].

- Update `_brightness`; call backlight callback.
- `_extend_dismiss()`.

```
def _extend_dismiss(self) -> None:
    if not _paused:
        _dismiss_at = time.monotonic() + OVERLAY_DISMISS
```

3.5 `piframe/settings_panel.py` — `SettingsPanel`

Renders the full-screen settings panel (sidebar + content area). Delegates section content to section helpers. Dispatches all widget interactions.

Sections

```
class Section(Enum):
    SLIDESHOW = "Slideshow"
    DISPLAY   = "Display"
    WIFI      = "Wi-Fi"
    SYSTEM    = "System"
```

Data members

Member	Type	Description
`_active_section`	`Section`	Current section
`_nav_items`	`list[NavItem]`	Sidebar nav rows
`_widgets`	`dict[Section, list[Widget]]`	Per-section widgets
`_scroll_y`	`int`	Content area scroll offset (px)
`_sync_status`	`SyncStatus`	Last known sync info
`_update_result`	`UpdateResult | None`	Latest OTA check result

`draw(screen)`

1. Draw sidebar: `pygame.draw.rect(screen, COLOUR_SIDEBAR_BG, (0,0,SIDEBAR_W,SCREEN_H))`.
2. Draw back button row `(0,0,SIDEBAR_W,58)`: `IC_ARROW_BACK` + label "Back to frame".
3. Draw `_nav_items` (y starting at 66, each 56 px tall).
4. Draw content area: `pygame.draw.rect(screen, COLOUR_CONTENT_BG, (SIDEBAR_W,0,SETTINGS_CONTENT_W,SCREEN_H))`.
5. Call `_draw_section(screen)` for `_active_section`.

Sidebar NavItem y-positions

Section	y_top	y_bottom
Slideshow	66	122
Display	122	178
Wi-Fi	178	234
System	234	290

Content area layout

Content starts at x=351 (333+18 padding), y=18 from content top. Row height: 52 px. Row label: 18pt at x=351. Control right-aligned to x=1262. Section title: 24pt NotoSans-Bold at y_rel=18. Divider: 1 px `COLOUR_DIVIDER`.

Slideshow section rows

Row label	Control	Config key
Interval	`SegmentedControl` (5s / 15s / 30s / 1m / 5m)	`slideshow.interval`
Fit mode	`SegmentedControl` (Fit / Fill)	`slideshow.fit_mode`
Shuffle	`Toggle`	`slideshow.shuffle`
Transition	`SegmentedControl` (Crossfade / Cut / Slide)	`slideshow.transition`

Display section rows

Row label	Control	Config key
Brightness	Horizontal `VerticalSlider`	`display.brightness`
Show clock	`Toggle`	`display.show_clock`
Sleep schedule	`Toggle`	`sleep.enabled`
Sleep time	`TimePicker` (conditional)	`sleep.sleep_time`
Wake time	`TimePicker` (conditional)	`sleep.wake_time`
Timezone	Current tz label + tap-to-open `ScrollPicker`	`system.timezone`

Wi-Fi section

1. Status row: connected SSID + IP, or "Not connected". Icon: `IC_WIFI` / `IC_WIFI_OFF`.
2. "Scan for networks" button → `WifiManager.scan()`.
3. Scrollable `WifiListItem` list (populated on scan result).
4. Connecting flow: if selected network has security → show `TextInput` for password → `Keyboard` opens → on Done → `WifiManager.connect(ssid, password)`.

System section rows

Row label	Control / Action
Sync status	Read-only label (last sync time, photo count)
Sync now	Button → `SyncService.trigger()`
Check for update	Button → `_check_update_async()`
Update available	Label + "Install" button (only if `_update_result.available`)
Reboot	Tap → `ConfirmDialog` → `subprocess.run(["sudo","reboot"])`
Shutdown	Tap → `ConfirmDialog` → `subprocess.run(["sudo","shutdown","-h","now"])`

`on_tap(pos) -> bool`

Returns `True` if tap was consumed. Delegates to active section widgets and sidebar nav items.

`on_wifi_result(result: WifiResult)`

Dispatched from App when `EVT_WIFI_RESULT` is received. Updates Wi-Fi section display (scan results list, connect success/failure, status row).

`on_update_result(result: UpdateResult)`

Stores `_update_result`; refreshes System section display.

`refresh_sync_status()`

Called by App on `EVT_SYNC_COMPLETE`. Re-reads `SyncService.status`.

`_check_update_async()`

```
def _worker():
    try:
        tag, url = check_update(config.update.repo)
        result = UpdateResult(available=True, tag_name=tag,
tarball_url=url)
    except Exception as e:
        result = UpdateResult(available=False, error=str(e))
    evt = pygame.event.Event(EVT_UPDATE_RESULT, result=result)
    pygame.event.post(evt)
threading.Thread(target=_worker, daemon=True).start()
```

`_apply_update_async(tarball_url)`

Spawns a daemon thread; calls `apply_update(tarball_url)`; on success calls `app.restart()`; on failure posts `EVT_UPDATE_RESULT` with error string.

3.6 `piframe/keyboard.py` — `Keyboard`

Full-width on-screen keyboard. Renders at y=450–800 (350 px tall) when visible. Supports alpha, numeric, and extended (#+=) layers with shift.

Layers

```
ALPHA = [
    list("QWERTYUIOP"),
    list("ASDFGHJKL"),
    ["SHIFT"] + list("ZXCVBNM") + ["BACKSPACE"],
    ["123", "SPACE", "DONE"],
]
NUMERIC = [
    list("1234567890"),
    list("-/:;()$&@"),
    ["#+="] + list(".,?!'") + ["BACKSPACE"],
    ["ABC", "SPACE", "DONE"],
]
EXTENDED = [
    list("[]{}#%^*+="),
    list(r"_\|~<>€£¥"),
    ["123"] + list(".,?!'") + ["BACKSPACE"],
    ["ABC", "SPACE", "DONE"],
]
```

Data members

Member	Type	Description
`_layer`	`str`	`"alpha"` | `"numeric"` | `"extended"`
`_shift`	`bool`	Single-shot shift active
`_target`	`TextInput | None`	Field receiving input
`_visible`	`bool`	—
`_key_rects`	`list[list[pygame.Rect]]`	Hit-test rects per row
`_active_key`	`tuple[int,int] | None`	(row, col) of pressed key

Key geometry (pre-computed)

```
Keyboard y start: 450; total height: 350 px
Top padding: 12 px; bottom padding: 12 px; row gap: 8 px
```

```
Row height: 75 px  (4 × 75 + 3 × 8 = 324; fits in 326 available)

Row 0 (10 keys): x_start=12,  key_w=122, gap=4
Row 1 ( 9 keys): x_start=75,  key_w=122, gap=4
Row 2:  SHIFT w=156 @ x=41; 7 alpha keys w=122; BACKSPACE w=156
Row 3:  "123"/ABC w=160 @ x=8; SPACE w=936 @ x=172; Done w=160 @ x=1112

Row y tops:  Row 0: 462  Row 1: 545  Row 2: 628  Row 3: 711
```

attach(target: TextInput)

```
_target  = target
_visible = True
_layer   = "alpha"
_shift   = False
```

detach()

```
_target  = None
_visible = False
```

draw(screen)

1. Fill keyboard background rect `(0, 450, SCREEN_W, 350)` with `COLOUR_SIDEBAR_BG`.
2. For each key in the current layer: a. Background: `COLOUR_KEY_BG_ACTIVE` if active; `COLOUR_KEY_BG_SPECIAL` for SHIFT / BACKSPACE / 123 / ABC / DONE; else `COLOUR_KEY_BG`. b. Draw rounded rect (radius=8). c. Draw key label centred (16pt font, or icon for BACKSPACE / SHIFT).

handle_event(event) -> bool

- `MOUSEBUTTONDOWN`: find (row, col) hit in `_key_rects`; set `_active_key`.
- `MOUSEBUTTONUP`: if `_active_key`: call `_emit(row, col)`; clear `_active_key`.
- Returns `True` if event consumed.

_emit(row, col)

```
key = current_layer_keys[row][col]
match key:
    case "BACKSPACE": _target.backspace()
    case "SPACE":     _target.append(" ")
    case "DONE":      detach(); on_done_callback()
    case "SHIFT":     _shift = not _shift
    case "123":       _layer = "numeric";  _shift = False
```

```
        case "#+=":       _layer = "extended"; _shift = False
        case "ABC":       _layer = "alpha";    _shift = False
        case _:
            ch = key.upper() if _shift else key.lower()
            _target.append(ch)
            if _shift: _shift = False  # single-shot
```

3.7 `piframe/clock_widget.py` — `ClockWidget`

Purpose

Renders a clock at top-left with time (48pt) and date (18pt). Uses a daemon thread to wake exactly at each minute boundary; main-thread render cost is zero between ticks.

Data members

Member	Type	Description
`_time_surf`	`pygame.Surface | None`	Pre-rendered time string
`_date_surf`	`pygame.Surface | None`	Pre-rendered date string
`_dirty`	`bool`	Set by ticker thread; cleared after draw
`_lock`	`threading.Lock`	Protects surfaces and dirty flag
`_stop_event`	`threading.Event`	Signals thread shutdown
`_timezone`	`datetime.timezone`	From config

Ticker thread algorithm

```
def _ticker(self):
    while not _stop_event.is_set():
        now = datetime.datetime.now(_timezone)
        _render_surfaces(now)
        with _lock: _dirty = True
        seconds_until = 60 - now.second
        _stop_event.wait(timeout=seconds_until)
```

`_render_surfaces(now)`

```
with _lock:
    time_str = now.strftime("%-I:%M %p")   # e.g. "3:04 PM"
    date_str = now.strftime("%A, %B %-d")  # e.g. "Monday, January 6"
    _time_surf = assets.font_bold(FONT_SIZE_CLOCK).render(time_str,
COLOUR_CLOCK_TEXT)
```

```
    _date_surf = assets.font(FONT_SIZE_BODY).render(date_str,
COLOUR_TEXT_SECONDARY)
```

`draw(screen)`

```
with _lock:
    if not _time_surf: return
    # Drop shadow: offset (+2, +2) in black at alpha 120
    shadow = pygame.Surface(_time_surf.get_size(), pygame.SRCALPHA)
    shadow.blit(_time_surf, (0,0))
    shadow.set_alpha(120)
    screen.blit(shadow, (16, 16))
    screen.blit(_time_surf, (14, 14))
    screen.blit(_date_surf, (14, 14 + _time_surf.get_height() + 4))
    _dirty = False
```

`update_timezone(tz: datetime.timezone)`

Sets `_timezone`. The ticker picks it up on its next wake.

`stop()`

```
_stop_event.set()
```

3.8 `piframe/sync_service.py` — `SyncService`

Purpose

Runs `framesync.sync_folder()` in a daemon thread on a configurable interval. Posts `EVT_SYNC_COMPLETE` on success. Exposes `trigger()` for manual sync.

Data members

Member	Type	Description
`_stop_event`	`threading.Event`	Shutdown signal
`_trigger_event`	`threading.Event`	Manual trigger signal
`_status`	`SyncStatus`	Protected by `_status_lock`
`_status_lock`	`threading.Lock`	—
`_interval_s`	`int`	Seconds between auto-syncs

Thread loop

```
def _run(self):
    while not _stop_event.is_set():
        _do_sync()
        remaining = _interval_s
        while remaining > 0 and not _stop_event.is_set():
            triggered = _trigger_event.wait(timeout=min(remaining, 60))
            if triggered:
                _trigger_event.clear()
                break
            remaining -= 60
```

`_do_sync()`

```
with _status_lock:
    _status.in_progress = True
try:
    from framesync import sync_folder, load_config
    cfg = load_config()
    sync_folder(cfg)
    with _status_lock:
        _status.last_sync_time = datetime.datetime.now()
        _status.in_progress    = False
        _status.last_error     = None
        _status.photo_count    =
len(list(Path(cfg["output_dir"]).glob("*.jp*g")))
    pygame.event.post(pygame.event.Event(EVT_SYNC_COMPLETE))
except Exception as e:
    with _status_lock:
        _status.in_progress = False
        _status.last_error  = str(e)
    logging.error("SyncService error: %s", e)
```

`trigger()`

```
_trigger_event.set()
```

`stop()`

```
_stop_event.set()
_trigger_event.set()  # unblock wait
```

`status -> SyncStatus`

```
    with _status_lock:
        return copy.copy(_status)
```

3.9 `piframe/sleep_scheduler.py` — `SleepScheduler`

Purpose

Background daemon thread. Polls the sleep window every 30 s. Posts `EVT_SLEEP` and `EVT_WAKE` as the current time enters or leaves the sleep window.

Data members

Member	Type	Description
`_stop_event`	`threading.Event`	Shutdown signal
`_sleeping`	`bool`	Tracked sleep state
`_grace_until`	`float`	`time.monotonic()` until grace period ends
`_config`	`ConfigStore`	Live config reference

Thread loop

```
    def _run(self):
        while not _stop_event.wait(timeout=30):
            cfg      = _config.sleep
            now_t    = datetime.datetime.now().time()
            in_grace = time.monotonic() < _grace_until

            if not cfg.enabled or in_grace:
                if _sleeping:
                    _sleeping = False
                    pygame.event.post(pygame.event.Event(EVT_WAKE))
                continue

            should_sleep = is_sleep_time(now_t, cfg.sleep_time_parsed,
    cfg.wake_time_parsed)
            if should_sleep and not _sleeping:
                _sleeping = True
                pygame.event.post(pygame.event.Event(EVT_SLEEP))
            elif not should_sleep and _sleeping:
                _sleeping = False
                pygame.event.post(pygame.event.Event(EVT_WAKE))
```

`is_sleep_time(now, sleep_t, wake_t) -> bool`

```
def is_sleep_time(now, sleep_t, wake_t):
    now_m   = now.hour     * 60 + now.minute
    sleep_m = sleep_t.hour * 60 + sleep_t.minute
    wake_m  = wake_t.hour  * 60 + wake_t.minute
    if sleep_m == wake_m: return False
    if sleep_m < wake_m:  return sleep_m <= now_m < wake_m
    return now_m >= sleep_m or now_m < wake_m  # midnight-crossing
```

`set_grace(until: float)`

```
_grace_until = until
```

`stop()`

```
_stop_event.set()
```

3.10 `piframe/config_store.py` — `ConfigStore`

Purpose

Reads `config.toml` at startup. Exposes a typed API for all config keys. Debounces writes: a 0.5 s quiet period after the last `set()` triggers a flush. Protected keys are never overwritten.

TOML schema

```
[slideshow]
interval    = 30
fit_mode    = "fit"
shuffle     = true
transition  = "crossfade"

[display]
brightness    = 80
show_clock    = true
timezone_auto = true

[sleep]
enabled    = false
sleep_time = "22:00"
wake_time  = "07:00"

[sync]
share_url        = ""
```

```
password         = ""
output_dir       = "/home/frame/Pictures/slideshow"
cache_dir        = "/home/frame/.cache/framesync"
interval_minutes = 60

[system]
timezone = "America/Los_Angeles"

[update]
repo = "njurgens/digital-frame"
```

Protected keys (never overwritten by `flush()`)

`sync.share_url`, `sync.password`, `sync.output_dir`, `sync.cache_dir`

Data members

Member	Type	Description
`_path`	`Path`	Path to `config.toml`
`_data`	`dict`	Parsed TOML (nested dict)
`_dirty_at`	`float | None`	Monotonic time of first dirty set; `None` if clean

`__init__(path)`

```
with open(path, "rb") as f:
    _data = tomllib.load(f)
_apply_defaults()
```

`tick(now: float)`

```
if _dirty_at and now - _dirty_at >= 0.5:
    flush_now()
```

`flush_now()`

```
disk = _read_raw()
for key in ("share_url", "password", "output_dir", "cache_dir"):
    _data["sync"][key] = disk.get("sync", {}).get(key, _data["sync"][key])
_write_toml(_data)
_dirty_at = None
```

`set(section, key, value)`

```
    _data[section][key] = value
    if _dirty_at is None:
        _dirty_at = time.monotonic()
```

Typed property accessors

```
    @property
    def slideshow(self) -> _SlideshowCfg: ...
    @property
    def display(self) -> _DisplayCfg: ...
    @property
    def sleep(self) -> _SleepCfg: ...
    @property
    def sync(self) -> _SyncCfg: ...
    @property
    def system(self) -> _SystemCfg: ...
    @property
    def update(self) -> _UpdateCfg: ...
```

`_SleepCfg` additionally exposes `sleep_time_parsed` and `wake_time_parsed` as `datetime.time` objects parsed from the `"HH:MM"` strings.

TOML writer (`_write_toml`)

Manual f-string serialiser (the schema is shallow, avoiding a pip dependency on `tomli_w`):

```
def _write_toml(data: dict) -> None:
    lines = []
    for section, values in data.items():
        lines.append(f"[{section}]")
        for k, v in values.items():
            if isinstance(v, bool):  lines.append(f"{k} =
{str(v).lower()}")
            elif isinstance(v, int): lines.append(f"{k} = {v}")
            elif isinstance(v, str): lines.append(f'{k} = "{v}"')
        lines.append("")
    _path.write_text("\n".join(lines))
```

3.11 `piframe/backlight.py` — `BacklightController`

Constants

```
BACKLIGHT_PATH = "/sys/class/backlight/10-0045/brightness"
MAX_VALUE      = 255
```

Data members

Member	Type	Description
`_last_percent`	`int | None`	Avoids redundant sysfs writes

`set_brightness(percent: int) -> None`

```
percent = max(0, min(100, percent))
if percent == _last_percent: return
sysfs_val = max(0, min(MAX_VALUE, round(percent / 100 * MAX_VALUE)))
try:
    with open(BACKLIGHT_PATH, "w") as f:
        f.write(f"{sysfs_val}\n")
    _last_percent = percent
except OSError as e:
    logging.warning("backlight write failed: %s", e)
```

`get_brightness() -> int`

```
try:
    with open(BACKLIGHT_PATH) as f:
        raw = int(f.read().strip())
    return round(raw / MAX_VALUE * 100)
except OSError:
    return 50
```

3.12 `piframe/wifi_manager.py` — `WifiManager`

Purpose

Wraps `nmcli` calls in daemon threads. All results posted via `EVT_WIFI_RESULT`.

Threading model

Each public method spawns one daemon thread. The thread runs the nmcli command, wraps the result in `WifiResult`, and calls:

```
pygame.event.post(pygame.event.Event(EVT_WIFI_RESULT, result=r))
```

Timeouts: `scan` = 10 s; `connect` = 15 s; all others = 5 s. All `nmcli` calls prefixed with `sudo` (frame user has `NOPASSWD: ALL`).

`scan()`

```
cmd = "sudo nmcli -t -f SSID,SECURITY,SIGNAL dev wifi list"
# Parse: each line → split(":") → [ssid, security, signal_str]
# Returns WifiResult(operation="scan", data=list[WifiNetwork])
```

`connect(ssid: str, password: str | None)`

```
if password:
    cmd = f"sudo nmcli dev wifi connect {ssid!r} password {password!r}"
else:
    cmd = f"sudo nmcli dev wifi connect {ssid!r}"
```

`forget(ssid: str)`

```
cmd = f"sudo nmcli connection delete {ssid!r}"
```

`disconnect()`

```
cmd = "sudo nmcli dev disconnect wlan0"
```

`status()`

Posts `WifiResult(operation="status", data=WifiStatus(...))`:

```
cmd = "sudo nmcli -t -f GENERAL.CONNECTION,IP4.ADDRESS device show wlan0"
# Parse key:value pairs; connected = GENERAL.CONNECTION not empty/"--"
```

`_run_cmd(cmd: str, timeout: int) -> tuple[bool, str]`

```
result = subprocess.run(
    cmd, shell=True, capture_output=True, text=True, timeout=timeout
```

```
    )
    return result.returncode == 0, result.stdout + result.stderr
```

3.13 `piframe/assets.py` — `Assets` singleton

Purpose

Central loader for all fonts and icon surfaces. Loaded once at startup by `App`.

Data members

```
_font_regular: pygame.freetype.Font   # NotoSans-Regular.ttf
_font_bold:    pygame.freetype.Font   # NotoSans-Bold.ttf
_icon_font:    pygame.freetype.Font   # MaterialIcons-Regular.ttf
```

`load()` (called once by App)

```
Assets._font_regular = pygame.freetype.Font("piframe/assets/fonts/NotoSans-
Regular.ttf")
Assets._font_bold    = pygame.freetype.Font("piframe/assets/fonts/NotoSans-
Bold.ttf")
Assets._icon_font    =
pygame.freetype.Font("piframe/assets/fonts/MaterialIcons-Regular.ttf")
```

`font(size: int) -> pygame.freetype.Font`

Returns `_font_regular` with `.size = size`.

`font_bold(size: int) -> pygame.freetype.Font`

Returns `_font_bold` with `.size = size`.

`icon(codepoint: str, size: int) -> pygame.Surface`

```
surf, _ = _icon_font.render(codepoint, fgcolor=(255,255,255,255),
size=size)
return surf
```

> `pygame.freetype.Font.render()` returns `(surface, rect)`.

---

4. Widget designs

All widgets inherit from `Widget` (`piframe/piframe/widgets/base.py`):

```
class Widget:
    def __init__(self, rect: pygame.Rect) -> None:
        self.rect = rect

    def draw(self, screen: pygame.Surface) -> None: ...
    def handle_event(self, event: pygame.event.Event) -> bool:
        """Return True if event consumed."""
        return False
    def update(self, dt: float) -> None: ...
```

4.1 `Toggle` (`piframe/widgets/toggle.py`)

Pixel spec

Property	Value
Track size	50 × 28 px
Track border-radius	14 px
Thumb diameter	22 px
Thumb travel	22 px (off → on)
Off thumb center-x	`rect.x + 14`
On thumb center-x	`rect.x + 36`
Animation duration	120 ms

Data members

Member	Type	Description
`_on`	`bool`	Current logical state
`_anim_t`	`float`	Animation position 0.0 → 1.0
`_speed`	`float`	Units/sec = 1.0/0.12 ≈ 8.33
`on_change`	`Callable[[bool], None] | None`	Change callback

`update(dt)`

```
target = 1.0 if _on else 0.0
if _anim_t != target:
    delta = _speed * dt
```

```
        if _on: _anim_t = min(1.0, _anim_t + delta)
        else:   _anim_t = max(0.0, _anim_t - delta)
```

`draw(screen)`

1. Lerp track colour between `COLOUR_TOGGLE_OFF` and `COLOUR_TOGGLE_ON` using `_anim_t`.
2. `pygame.draw.rect(screen, track_colour, rect, border_radius=14)`.
3. Thumb center-x: `rect.x + 14 + int(_anim_t * 22)`.
4. `pygame.draw.circle(screen, COLOUR_TOGGLE_THUMB, center, 11)`.

`handle_event(event) -> bool`

On `MOUSEBUTTONDOWN` in `rect`:

```
_on = not _on
if on_change: on_change(_on)
return True
```

4.2 `VerticalSlider` (`piframe/widgets/vertical_slider.py`)

Pixel spec

Property	Value
Track width	4 px (centered in widget rect)
Track colour	`COLOUR_SLIDER_TRACK`
Fill colour	`COLOUR_SLIDER_FILL`
Thumb diameter	22 px
Thumb colour	`COLOUR_SLIDER_THUMB`
Value range	0–100

Value ↔ y conversion

```
def _value_to_y(self, value: int) -> int:
    return self.rect.top + 11 + int((1.0 - value / 100) * (self.rect.height
- 22))

def _y_to_value(self, y: int) -> int:
    raw = 1.0 - (y - self.rect.top - 11) / (self.rect.height - 22)
    return round(max(0.0, min(1.0, raw)) * 100)
```

`draw(screen)`

1. Track rect: `pygame.Rect(rect.centerx - 2, rect.top, 4, rect.height)`.
2. Draw full track with `COLOUR_SLIDER_TRACK`.
3. Draw fill from `_value_to_y(value)` to `rect.bottom` with `COLOUR_SLIDER_FILL`.
4. Draw thumb circle at `(rect.centerx, _value_to_y(value))`.

`handle_event(event) -> bool`

- `MOUSEBUTTONDOWN` in `rect.inflate(20, 0)`: start drag, update value.
- `MOUSEMOTION` while dragging: update value, call `on_change(value)`.
- `MOUSEBUTTONUP`: end drag.

4.3 `SegmentedControl` (`piframe/widgets/segmented_control.py`)

Horizontal row of 2–4 labelled segments.

Pixel spec

Property	Value
Height	36 px
Segment border-radius	8 px
Selected bg	`COLOUR_BTN_PRIMARY`
Unselected bg	`COLOUR_BTN_SECONDARY`
Label	14pt NotoSans

Data members

Member	Type
`_segments`	`list[str]`
`_selected`	`int`
`on_change`	`Callable[[int, str], None] | None`

`draw(screen)`

1. `seg_w = rect.width // len(_segments)`.
2. For each segment: draw rounded rect background; draw centred label.

`handle_event(event) -> bool`

`MOUSEBUTTONDOWN` → find segment index → `_selected = i` → `on_change(i, _segments[i])`.

4.4 `ScrollPicker` (`piframe/widgets/scroll_picker.py`)

Scrollable list for timezone and time selection.

Pixel spec

Property	Value
Visible rows	7
Row height	44 px
Widget height	308 px
Highlight strip	`COLOUR_SCROLL_PICKER_HL` (centre row)
Text	18pt NotoSans

Data members

Member	Type	Description
`_items`	`list[str]`	All items
`_scroll_offset`	`float`	Fractional index of top visible row
`_surface_cache`	`dict[int, pygame.Surface]`	Text surfaces by row index
`on_change`	`Callable[[int, str], None] | None`	—
`_drag_y`	`int | None`	y at MOUSEBUTTONDOWN
`_drag_offset`	`float`	`_scroll_offset` at drag start

`draw(screen)`

1. Clip to `rect`.
2. Draw highlight strip at centre row y position.
3. Compute `first = int(_scroll_offset)`; render rows `first` to `first + 8`.
4. For each visible row: `y_top = rect.top + (row_idx - _scroll_offset) * ROW_H`.
5. Draw text surface (from cache; evict if index > `visible_rows * 3` from window).
6. Fade top and bottom edges with alpha gradient.

`handle_event(event) -> bool`

- `MOUSEBUTTONDOWN` in rect: record `_drag_y`, `_drag_offset`.
- `MOUSEMOTION` while dragging: `_scroll_offset = _drag_offset - (y - _drag_y) / ROW_H`; clamp to `[0, len(_items) - visible_rows]`.
- `MOUSEBUTTONUP`: snap to nearest integer; call `on_change`.

4.5 `TimePicker` (`piframe/widgets/time_picker.py`)

Two pill buttons (HH, MM) that open a popup with two `ScrollPicker` columns.

Pixel spec

Property	Value
Pill size	80 × 44 px minimum
Pill bg	`COLOUR_PILL_BG`
Pill border	`COLOUR_PILL_BORDER`
Popup size	320 × 280 px
Popup bg	`COLOUR_DIALOG_BG`
Popup border	`COLOUR_DIALOG_BORDER`, 1 px, radius 8 px
Hour picker	Hours 0–23, left column
Minute picker	Minutes 0–59, right column
Done button	32 × 32 px at popup top-right

Data members

Member	Type	Description
`_hour`	`int`	0–23
`_minute`	`int`	0–59
`_popup_open`	`bool`	—
`_popup_rect`	`pygame.Rect`	Computed on open
`_hour_picker`	`ScrollPicker`	—
`_min_picker`	`ScrollPicker`	—
`on_change`	`Callable[[int, int], None] | None`	Called on Done

Popup placement

```
popup_y = rect.bottom + 4
if popup_y + 280 > SCREEN_H:
    popup_y = rect.top - 280 - 4
popup_x = max(0, min(rect.centerx - 160, SCREEN_W - 320))
_popup_rect = pygame.Rect(popup_x, popup_y, 320, 280)
```

`draw(screen)`

1. Draw two pill buttons formatted as `f"{_hour:02d}"` and `f"{_minute:02d}"`.

2. If `_popup_open`: draw popup panel; both `ScrollPicker`s; Done button.

`handle_event(event) -> bool`

- Tap on either pill: open popup, pre-scroll pickers to current values.
- Tap outside popup while open: close without change.
- Done tap: update `_hour`, `_minute`; call `on_change`; close popup.
- Delegate events to pickers while popup open.

4.6 `WifiListItem` (`piframe/widgets/wifi_list_item.py`)

Single row in the Wi-Fi network list.

Pixel spec

Property	Value
Row height	56 px
Signal icon	24 px (3 strength tiers)
Lock icon	16 px (shown if `security != ""`)
SSID label	18pt
Security caption	14pt `COLOUR_TEXT_SECONDARY`
Connected dot	8 px `COLOUR_CONNECTED`

`draw(screen)`

1. Row background: `COLOUR_NAV_ACTIVE_BG` if selected, else transparent.
2. Wi-Fi signal icon at appropriate tier.
3. Lock icon if `network.security != ""`.
4. SSID text.
5. Security/frequency caption.
6. Connected dot if this SSID matches current connection.

4.7 `ConfirmDialog` (`piframe/widgets/confirm_dialog.py`)

Blocking modal dialog.

Pixel spec

Property	Value
Size	480 × 240 px
Position	`(400, 280)` (centred on 1280×800)
Background	`COLOUR_DIALOG_BG`

Property	Value
Border	1 px `COLOUR_DIALOG_BORDER`, radius 12 px
Title	24pt NotoSans-Bold, y_rel=30
Body text	18pt NotoSans, y_rel=72
Cancel button	196 × 52 px at `(20, 168)`, `COLOUR_BTN_SECONDARY`
Confirm button	196 × 52 px at `(264, 168)`, `COLOUR_DESTRUCTIVE` or `COLOUR_BTN_PRIMARY`

Data members

Member	Type	Description
`title`	`str`	—
`body`	`str`	—
`confirm_label`	`str`	Default `"Confirm"`
`destructive`	`bool`	Uses `COLOUR_DESTRUCTIVE` if `True`
`on_confirm`	`Callable[[], None]`	—
`on_cancel`	`Callable[[], None]`	—

`draw(screen)`

1. Draw full-screen semi-transparent scrim `COLOUR_SCRIM`.
2. Draw dialog background rect at `(400, 280, 480, 240)`.
3. Draw title and body text.
4. Draw Cancel and Confirm buttons.

`handle_event(event) -> bool`

- Tap Cancel rect → `on_cancel()`; return True.
- Tap Confirm rect → `on_confirm()`; return True.
- Tap outside → `on_cancel()`; return True.
- Always consumes the event when visible.

4.8 `NavItem` (`piframe/widgets/nav_item.py`)

Single sidebar navigation row.

Pixel spec

Property	Value
Row height	56 px

Property	Value
Icon center-x	28 px
Label x	56 px
Label size	20pt NotoSans
Active background	`COLOUR_NAV_ACTIVE_BG`
Active icon colour	`COLOUR_CONNECTED`
Active text	`COLOUR_TEXT_PRIMARY`
Inactive text	`COLOUR_TEXT_SECONDARY`

`draw(screen)`

1. If active: fill row rect with `COLOUR_NAV_ACTIVE_BG`.
2. Draw icon at `(28, rect.centery)` using `COLOUR_CONNECTED` if active, else `COLOUR_TEXT_SECONDARY`.
3. Draw label at `(56, rect.centery)`.

`handle_event(event) -> bool`

`MOUSEBUTTONDOWN` in `rect` → `active = True` → call `on_select()`.

4.9 `TextInput` (`piframe/widgets/text_input.py`)

Single-line text field. The `Keyboard` attaches to it.

Data members

Member	Type	Description
`_text`	`str`	Current value
`_placeholder`	`str`	Shown when empty
`_focused`	`bool`	—
`on_focus`	`Callable[[], None] | None`	Called when tapped
`on_change`	`Callable[[str], None] | None`	Called on each edit
`_password_mode`	`bool`	Renders `*` per character

`append(ch: str)`

```
_text += ch
if on_change: on_change(_text)
```

`backspace()`

```
_text = _text[:-1]
if on_change: on_change(_text)
```

`draw(screen)`

1. Draw border rect (1 px `COLOUR_DIVIDER`; `COLOUR_BTN_PRIMARY` if focused).
2. If empty: draw placeholder in `COLOUR_TEXT_CAPTION`.
3. Else: draw text (`*` per char if `_password_mode`).
4. If focused: draw cursor at text end.

`handle_event(event) -> bool`

`MOUSEBUTTONDOWN` in `rect`: `_focused = True`; call `on_focus()`; return True. Click outside: `_focused = False`; return False.

---

5. Asset specification

5.1 Fonts

File	License	Usage
`piframe/piframe/assets/fonts/NotoSans-Regular.ttf`	OFL 1.1	Body, labels, keyboard keys, captions
`piframe/piframe/assets/fonts/NotoSans-Bold.ttf`	OFL 1.1	Clock time, section headings, dialog titles
`piframe/piframe/assets/fonts/MaterialIcons-Regular.ttf`	Apache 2.0	All icon glyphs

Font sizes used (pt):

Size	Usage
48	Clock time
24	Section headings, dialog titles
20	Sidebar nav labels
18	Settings body text, overlay labels
16	Keyboard key labels
14	Secondary labels, captions, brightness percent

5.2 Icon codepoints

Constant	Codepoint	Material name	Usage
IC_SETTINGS	\ue8b8	settings	Overlay gear button
IC_PLAY	\ue037	play_arrow	Overlay play button
IC_PAUSE	\ue034	pause	Overlay pause button
IC_SKIP_PREV	\ue044	skip_previous	Overlay previous
IC_SKIP_NEXT	\ue043	skip_next	Overlay next
IC_ARROW_BACK	\ue5d5	arrow_back	Settings back; keyboard backspace
IC_ARROW_FWD	\ue5dc	arrow_forward	—
IC_INFO	\ue87d	info	Info overlay (deferred)
IC_WIFI	\ue8f4	wifi	Wi-Fi connected
IC_WIFI_OFF	\ue8f5	wifi_off	Wi-Fi disconnected
IC_SYNC	\ue1d8	sync	Sync status row
IC_CLOSE	\ue5cd	close	Popup close
IC_CHECK	\ue876	check	Confirm / connected indicator
IC_CHEVRON_L	\ue5c4	chevron_left	TimePicker
IC_CHEVRON_R	\ue5c8	chevron_right	TimePicker
IC_EXPAND_MORE	\ue5cf	expand_more	Dropdown expand
IC_EXPAND_LESS	\ue5ce	expand_less	Dropdown collapse
IC_BRIGHTNESS	\ue896	brightness_6	Brightness icons
IC_SCHEDULE	\ue8b5	schedule	Sleep schedule row
IC_PERSON	\ue7ef	person	Account / owner row
IC_DELETE	\ue872	delete	Forget network

5.3 Colour palette

See §2.5 for the complete `COLOUR_*` table. All colours are `(R, G, B, A)` tuples. Colours with A < 255 require surfaces created with `pygame.SRCALPHA`.

---

6. Per-stage implementation plans

Stage 1 — Fullscreen slideshow

Goal: Photo slideshow with crossfade/cut/slide, EXIF correction, blurred-background composite, clock overlay, shuffle, auto-advance.

Files to create: `slideshow.py`, `piframe/app.py` (App + SlideshowPlayer), `piframe/photo_cache.py`, `piframe/clock_widget.py`, `piframe/assets.py`, `piframe/types.py`, `piframe/config_store.py` (minimal), `piframe/piframe/assets/fonts/` (three TTFs).

Steps:

1. Create `piframe/types.py`: screen constants, colour palette, `AppState`, `AppEvent`, `EVT_*` IDs, `SyncStatus`.
2. Create `piframe/assets.py`: `Assets.load()`, `font()`, `font_bold()`, `icon()`.
3. Create `piframe/photo_cache.py`: `PhotoCache.__init__`, `get()`, fit/fill composites, EXIF correction, LRU `_put()`.
4. Create `piframe/clock_widget.py`: ticker thread, `_render_surfaces()`, `draw()`, `stop()`.
5. Create `piframe/app.py`: a. `SlideshowPlayer`: `rescan()`, `_fisher_yates()`, `update()`, `advance()`, `go_back()`, `draw()`, `draw_pip()`. b. `App.__init__` (no overlay/settings), `run()`, `_process_pygame_events()` (QUIT + KEYDOWN only), `_draw()` (layers 1–3).
6. Create `slideshow.py`: `from piframe.app import App; App().run()`.
7. Create `config.toml.example` with all keys defaulted.
8. **Verify:** photos cycle; clock visible; EXIF correct; blurred background on portrait images.

Open items resolved: OR-03 (cache key), OR-04 (slide direction), OR-08 (memory budget).

Stage 2 — Transient overlay

Goal: Tap-to-show overlay with play/pause, skip prev/next, brightness slider, settings gear. Swipe detection.

Files to create: `piframe/overlay_ui.py`, `piframe/backlight.py`, `piframe/piframe/widgets/base.py`, `piframe/widgets/vertical_slider.py`, `piframe/piframe/widgets/__init__.py`.

Files to modify: `piframe/app.py` (OVERLAY state, pointer dispatch, drag forwarding, overlay layer).

Steps:

1. Create `piframe/backlight.py`: `set_brightness()`, `get_brightness()`, sysfs path, `OSError` handling.
2. Create `piframe/piframe/widgets/base.py`: `Widget` ABC.
3. Create `piframe/widgets/vertical_slider.py`: full pixel spec, value↔y, drag handling.
4. Create `piframe/overlay_ui.py`: layout constants, `show()`, `hide()`, `update()`, `draw()`, `on_tap()`, `on_drag()`.
5. Add OVERLAY state to `piframe/app.py`:
   - `MOUSEBUTTONDOWN` in SLIDESHOW → `_overlay.show()`, `_state = OVERLAY`.
   - Pointer up → `_classify_pointer_up()` / `_dispatch_tap()`.
   - `_overlay.dismissed` check in `_update()`.
   - Brightness callback: `_overlay._slider.on_change = _backlight.set_brightness`.
6. **Verify:** tap shows overlay; slider changes brightness live; prev/next/play-pause work; overlay auto-dismisses after 5 s; swipe skips photos.

Stage 3 — TOML config + brightness persistence

Goal: Full `ConfigStore` with debounce; brightness, interval, fit_mode, shuffle, transition all persisted.

Files to create: `piframe/config_store.py` (complete).

Files to modify: `piframe/app.py`, `piframe/photo_cache.py`, `piframe/overlay_ui.py`.

Steps:

1. Implement `ConfigStore` in full: `__init__`, `tick()`, `flush_now()`, `set()`, typed accessors, `_write_toml()`, `_apply_defaults()`.
2. Thread `config` through `App.__init__`; pass to all sub-modules.
3. Add `_config.tick()` to `App.run()`.
4. Apply `_config.display.brightness` on startup via `_backlight.set_brightness()`.
5. Brightness change: `_config.set("display", "brightness", v)`.
6. `PhotoCache` reads `_config.slideshow.fit_mode` on each `get()`.
7. `SlideshowPlayer` reads `_config.slideshow.interval` each update.
8. **Verify:** change brightness via overlay, relaunch — brightness restored from TOML.

Stage 4 — Settings panel scaffold + Slideshow section

Goal: Settings panel opens from gear icon. Sidebar navigation works. Slideshow section fully functional.

Files to create: `piframe/settings_panel.py`, `piframe/widgets/segmented_control.py`, `piframe/widgets/toggle.py`, `piframe/widgets/nav_item.py`.

Files to modify: `piframe/app.py` (SETTINGS state, gear-tap, back-tap, settings layer).

Steps:

1. Create `piframe/widgets/toggle.py`: animation, `draw()`, `handle_event()`.
2. Create `piframe/widgets/segmented_control.py`: `draw()`, `handle_event()`.
3. Create `piframe/widgets/nav_item.py`: `draw()`, `handle_event()`.
4. Create `piframe/settings_panel.py`: a. Sidebar with back button and four NavItems. b. Content area with Slideshow section rows; each `on_change` → `_config.set(...)`.
5. Add SETTINGS state to `App`:
   - Gear tap → `_state = SETTINGS; _settings.open()`.
   - `_settings.on_tap()` in pointer dispatch.
   - Back button → `_state = SLIDESHOW`.
6. **Verify:** settings opens; all slideshow settings change and persist; navigation between sections shows placeholders.

Stage 5 — Display section + sleep schedule

Goal: Display section functional; sleep schedule; timezone picker.

Files to create: `piframe/sleep_scheduler.py`, `piframe/widgets/scroll_picker.py`, `piframe/widgets/time_picker.py`.

Files to modify: `piframe/app.py` (SleepScheduler, EVT_SLEEP/EVT_WAKE), `piframe/settings_panel.py` (Display section), `piframe/config_store.py` (sleep + system sections).

Steps:

1. Create `piframe/widgets/scroll_picker.py`: windowed rendering (OR-07), LRU text cache, drag scroll, snap on release.
2. Create `piframe/widgets/time_picker.py`: pill buttons, popup placement (OR-10), embedded `ScrollPicker` columns, Done button.
3. Create `piframe/sleep_scheduler.py`: daemon thread, `is_sleep_time()` (OR-05), `set_grace()`, EVT_SLEEP/EVT_WAKE.
4. Add Display section to `piframe/settings_panel.py` with all rows from §3.5.
5. Wire timezone change → `_config.set("system", "timezone", tz)` + `_clock_w.update_timezone()`.
6. Start `SleepScheduler` in `App.__init__`; handle `EVT_SLEEP`/`EVT_WAKE`.
7. **Verify:** sleep dims screen at configured time; tap wakes; timezone change reflected in clock immediately.

Stage 6 — On-screen keyboard

Goal: `Keyboard` widget functional; `TextInput`; KEYBOARD app state.

Files to create: `piframe/keyboard.py`, `piframe/widgets/text_input.py`.

Files to modify: `piframe/app.py` (KEYBOARD state), `piframe/settings_panel.py` (TextInput wiring).

Steps:

1. Create `piframe/widgets/text_input.py`: `append()`, `backspace()`, `draw()`, `handle_event()`, password-mode.
2. Create `piframe/keyboard.py`: key geometry constants, layer tables, `attach()`, `detach()`, `draw()`, `handle_event()`, `_emit()`, shift logic.
3. Add KEYBOARD state to `App`:
   - `TextInput.on_focus` → `_keyboard.attach(field)`, `_state = KEYBOARD`.
   - Outside keyboard rect tap → `_keyboard.detach()`, `_state = SETTINGS`.
   - `EVT_SLEEP` while in KEYBOARD → `_enter_sleep()`.
4. Add keyboard layer to `_draw()`.
5. **Verify:** tap a text field → keyboard appears; all three layers work; Done dismisses keyboard; result appears in field.

Stage 7 — Wi-Fi section

Goal: Wi-Fi section: scan, connect (with keyboard for password), forget, disconnect.

Files to create: `piframe/wifi_manager.py`, `piframe/widgets/wifi_list_item.py`.

Files to modify: `piframe/settings_panel.py` (Wi-Fi section), `piframe/app.py` (EVT_WIFI_RESULT).

Steps:

1. Create `piframe/wifi_manager.py`: all operations, daemon thread pattern, nmcli parse logic (OR-06).
2. Create `piframe/widgets/wifi_list_item.py`: signal tier icons, lock icon, connected dot.
3. Implement Wi-Fi section in `piframe/settings_panel.py`: a. Status row; scan button; WifiListItem list. b. Tap open network → `connect(ssid, None)`. c. Tap secured network → `TextInput` + keyboard → `connect(ssid, password)`. d. Long-press connected item → `ConfirmDialog` "Forget?" → `forget(ssid)`.
4. Handle `EVT_WIFI_RESULT` in `App._drain_custom_events()`.
5. **Verify:** scan lists networks; connect to open and WPA2 networks; forget works; status row updates.

Stage 8 — System section + OTA

Goal: System section: sync status, manual sync, OTA check+install, reboot/shutdown.

Files to create: `piframe/widgets/confirm_dialog.py`.

Files to modify: `piframe/settings_panel.py` (System section), `piframe/app.py` (EVT_UPDATE_RESULT, `restart()`).

Steps:

1. Create `piframe/widgets/confirm_dialog.py`: `draw()`, `handle_event()`, scrim.
2. Implement System section in `piframe/settings_panel.py` per §3.5.
3. Implement `check_update()` / `apply_update()` (OR-01).
4. Implement `App.restart()` (OR-02).
5. Handle `EVT_UPDATE_RESULT` → `_settings.on_update_result(result)`.
6. Integrate `ConfirmDialog` into `App.draw_dialogs()`.
7. **Verify:** OTA check contacts GitHub API; update installs and restarts; reboot/shutdown confirmations work.

Stage 9 — OneDrive sync integration

Goal: Replace framesync systemd units with in-process `SyncService`. Slideshow rescans after each sync.

Files to create: `piframe/sync_service.py`.

Files to modify: `piframe/app.py` (SyncService), `eng/install.sh` (retire units).

Steps:

1. Create `piframe/sync_service.py`: daemon thread, `_do_sync()` calling `framesync.sync_folder()`, `trigger()`, `stop()`, `status` property (OR-09).
2. Construct `SyncService` in `App.__init__`; start daemon thread.
3. Handle `EVT_SYNC_COMPLETE`:

```
_player.rescan()
_settings.refresh_sync_status()
```

4. Update sync status row in System section.
5. Add to `eng/install.sh`:

```
sudo systemctl disable --now framesync.service framesync.timer || true
```

6. Remove framesync unit file installation from `eng/install.sh`.
7. **Verify:** new photos appear in slideshow within one interval after `trigger()`; sync status row updates.

---

7. Open items resolution

OR-01: OTA Update Mechanism

Decision: Use GitHub Releases API with `urllib.request` (stdlib, no pip). Download tarball; extract with `tarfile`; `shutil.copytree` with `dirs_exist_ok=True`; skip `config.toml`, `assets/`, `.git`. On exception: log, clean up staging files.

```
GITHUB_API = "https://api.github.com/repos/{repo}/releases/latest"

def check_update(repo: str) -> tuple[str, str]:
    url = GITHUB_API.format(repo=repo)
    with urllib.request.urlopen(url, timeout=10) as r:
        data = json.loads(r.read())
    return data["tag_name"], data["tarball_url"]

def apply_update(tarball_url: str) -> None:
    staging_tar = "/tmp/pi-frame-update.tar.gz"
    staging_dir = "/tmp/pi-frame-update/"
    try:
        urllib.request.urlretrieve(tarball_url, staging_tar)
        with tarfile.open(staging_tar) as tf:
            tf.extractall(staging_dir)
        src = next(Path(staging_dir).iterdir())
        dst = Path(__file__).parent
        shutil.copytree(src, dst, dirs_exist_ok=True,

ignore=shutil.ignore_patterns("config.toml","assets",".git"))
    finally:
        Path(staging_tar).unlink(missing_ok=True)
        shutil.rmtree(staging_dir, ignore_errors=True)
```

Runs in daemon thread; result posted as `EVT_UPDATE_RESULT`. Implemented in Stage 8.

OR-02: App Restart Under labwc

Decision: Use `os.execve` to replace the process image in-place, preserving Wayland environment variables:

```
def restart(self) -> None:
    self._cleanup()
    env = os.environ.copy()
    env["XDG_RUNTIME_DIR"] = "/run/user/1000"
    env["WAYLAND_DISPLAY"]  = "wayland-0"
    os.execve(sys.executable, [sys.executable] + sys.argv, env)
```

`_cleanup()` stops background threads and flushes config before exec.

OR-03: Cache Key Includes `fit_mode`

Decision: Key format: `"{stem}_{fit_mode}_v{_CACHE_VERSION}"` where `_CACHE_VERSION = 2`. Changing `fit_mode` automatically misses old cached entries; no explicit invalidation needed.

Example keys: `IMG_0042_fit_v2.png`, `IMG_0042_fill_v2.png`.

OR-04: Slide Transition Direction

Decision: `_direction = +1` for forward; `-1` for go_back.

```
cur_x  = int(-_direction * _trans_progress * SCREEN_W)
next_x = int( _direction * (1.0 - _trans_progress) * SCREEN_W)
```

Forward: current slides left off-screen; next slides in from right. Backward: current slides right; next slides in from left.

OR-05: Midnight-Crossing Sleep Window

Decision: All arithmetic in total-minutes-since-midnight:

```
def is_sleep_time(now, sleep_t, wake_t) -> bool:
    now_m   = now.hour     * 60 + now.minute
    sleep_m = sleep_t.hour * 60 + sleep_t.minute
    wake_m  = wake_t.hour  * 60 + wake_t.minute
    if sleep_m == wake_m: return False
    if sleep_m < wake_m:  return sleep_m <= now_m < wake_m
    return now_m >= sleep_m or now_m < wake_m
```

Correctly handles sleep=22:00, wake=07:00 across midnight.

OR-06: nmcli and polkit

Decision: No polkit agent needed. The `frame` user has `NOPASSWD: ALL` in sudoers (existing install.sh). All nmcli calls prefixed with `sudo`. Verified on-device.

OR-07: Timezone Picker Windowing

Decision: `ScrollPicker` renders only rows near the current scroll offset.

```
visible_rows = 7
row_height   = 44 px
_scroll_offset: float (fractional index of top visible row)
first = int(_scroll_offset)
render rows [first, first + 8)
```

LRU eviction: evict cached row surfaces more than `visible_rows * 3 = 21` rows from the current window. Prevents unbounded growth on ~600 IANA timezone entries.

OR-08: Surface Memory Budget

Decision:

- All composited surfaces: 32-bit RGBA (4 bytes/pixel).
- Single surface: 1280 × 800 × 4 = ~4 MB.
- `MAX_CACHE = 6` → 24 MB peak in-memory cache.
- During crossfade: `_current_surface` (4 MB) + `_trans_incoming` copy (4 MB) + display buffer (~4 MB) ≈ 12 MB active.
- Total estimated app RSS: 80–100 MB — safe on 512 MB Pi 3A+.
- `_trans_incoming` is a **one-time copy at transition start**, not per frame. Set to `None` after commit.

OR-09: framesync systemd units retired

Decision: `framesync/framesync.py` is a modifiable module. `SyncService._do_sync()` calls `framesync.sync_folder()` directly after importing from the `framesync` package.

The `framesync.service` and `framesync.timer` units are disabled and stopped in Stage 9:

```
sudo systemctl disable --now framesync.service framesync.timer || true
```

Unit files removed from `eng/install.sh`; `framesync/framesync.py` kept in the repository.

OR-10: TimePicker Interaction Model

Decision: Two pill buttons (80 × 44 px). Tapping either opens a 320 × 280 px popup with two `ScrollPicker` columns (hours 0–23 left; minutes 0–59 right). Popup placement: prefer below the pill; place above if insufficient space below. A 32 × 32 px Done button at top-right of the popup confirms. Tapping outside closes without change.

Each column: 6 visible rows × 40 px = 240 px, fitting within the popup content area.

End of document.